

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks: 30

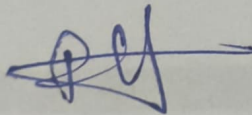
Annexure A

Dry Run Evaluation

Code of Conduct:

I Shaik. Fahed (192565026) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



20 + 25 + 18 + 18 + 8

89

10

1. Question 25 - Search Strategy Comparison

Consider a graph with $S \rightarrow A, B$, $A \rightarrow C, D$, $B \rightarrow E$, $C \rightarrow G$, $D \rightarrow G$, $E \rightarrow G$.

Perform BFS, DFS, UCS, Greedy Best-First Search and A* separately. Complete the comparison table.

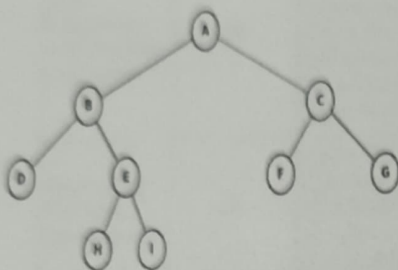
Algorithm	Expansion Order	Path	Cost	Complete?	Optimal?
1					
2					
3					
4					
5					

2. Question 1 - Breadth-First Search (BFS)

Consider the following graph.

Task: Starting from node A, perform Breadth-First Search (BFS).

Complete the table below.

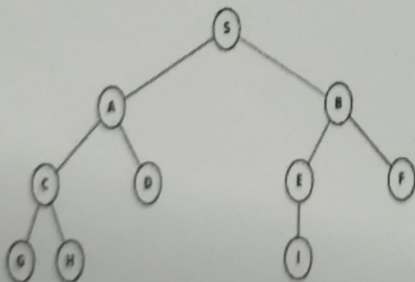


Iteration	Queue Before	Expanded	Queue After
1			
2			
3			
4			
5			

3. Question 2 - Depth-First Search (DFS)

Using the following graph, perform Depth-First Search (DFS) starting from node S.

Complete the table below.



Iteration	Stack Before	Expanded	Stack After
1			
2			
3			
4			
5			

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

20+25+18+18+8

89

DE

1. Search Strategy Comparison

Given graph:

$S \rightarrow A, B$

$A \rightarrow C, D$

$B \rightarrow E$

$C \rightarrow G$

$D \rightarrow G$

$E \rightarrow G$

Since no edge costs or heuristic values are provided, assume unit cost for every edge, for Greedy and A^* , no heuristic is given, so they cannot be uniquely calculated.

Algorithm

Expansion Order

Path to G

Cost

Complete

Optimal

BFS

S, A, B, C, D, E, G

S-A-C-G

3

Yes

Yes

DFS

S, A, C, G

S-A-C-G

3

Yes

Yes

UCS

S, A, B, C, D, E, G

S-A-C-G

3

Yes

Yes

Greedy BFS

can't w/o h(n)

depend on

No

A^*

can't determine

2. Question-1

Breadth-First search

(BFS)

Starting Node: A

BFS is an uninformed search algorithm that explores the graph level by level. It uses a FIFO queue, meaning the node inserted first is expanded first.

Steps:-

1) Start with A.

2) Expand A $\rightarrow$ add B, C

3) Expand B $\rightarrow$ add D, E

4) Expand C $\rightarrow$ add F, G

5) Expand D $\rightarrow$ no children

6) Expand E $\rightarrow$ add H, I

7) Continue until all nodes are visited.

Iteration	Queue Before	Expanded	Queue After
1.	A	A	B, C
2.	B, C	B	C, D, E
3.	C, D, E	C	D, E, F, G
4.	D, E, F, G	D	E, F, G
5.	E, F, G	E	F, G, H, I

BFS Expansion Order :

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I$

Result:-

BFS visits the nodes level by level and ensures that nodes at the same depth are explored before moving to the next level.

Question-3:-

Depth-First Search (DFS)

Starting node:- S

DFS explores one branch as deeply as possible before back tracking. It uses a LIFO stack, so the last node inserted is explored first.

Assuming nodes are visited left to right, the traversal is,

S → A → C → G → H → D → B → E → J → F

Steps:-

- 1) start with S → push A & B.
- 2) expand A → go deeper to C.
- 3) expand C → go deeper to G
- 4) G has no children → back track to C
- 5) visit H → back track A.

6) visit D $\rightarrow$ back track to S.

7) visit B $\rightarrow$ back track to S

8) visit B $\rightarrow$ Explore E

9) E $\rightarrow$ I

10) Finally, visit F.

Iteration	Stack Before	Expanded	Stack After
1.	S	S	A, B
2.	A, B	A	C, D, B
3.	C, D, B	C	G, H, D, B
4.	G, H, D, B	G	H, D, B
5.	H, D, B	H	D, B

DFS Expansion order:-

S $\rightarrow$ A $\rightarrow$ C $\rightarrow$ G $\rightarrow$ H $\rightarrow$ D $\rightarrow$ B $\rightarrow$ E
 $\rightarrow$ I $\rightarrow$ F

Result

DFS follows a path deeply untill it reaches a node with no unvisited children, then back tracks and explores the remaining branches.